LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH KOTLIN

Oleh:

Arya Arrozza Ridho Syaputra **NIM. 2410817210010**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Kotlin Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arya Arrozza Ridho Syaputra
NIM : 2410817210010

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 9
B. Output Program 14
C. Pembahasan 17
D. Essay 21
E. Daftar Pustaka 27
Tautan GIT 28

DAFTAR GAMBAR

Gambar 1.Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Dadu Setelah Di-Roll 7
Gambar 3. Tampilan Roll Dadu Double 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML 14
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML 15
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML 15
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose 16
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose 16
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose 17

DAFTAR TABEL

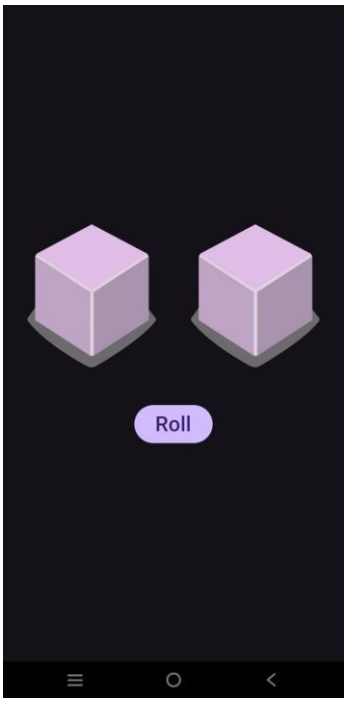
Tabel 1. Source Code MainActivity.Kt XML 9
Tabel 2. Source Code activity_main.xml 10
Tabel 3. Source Code MainActivity.Kt Compose 11

SOAL 1

Soal Praktikum:

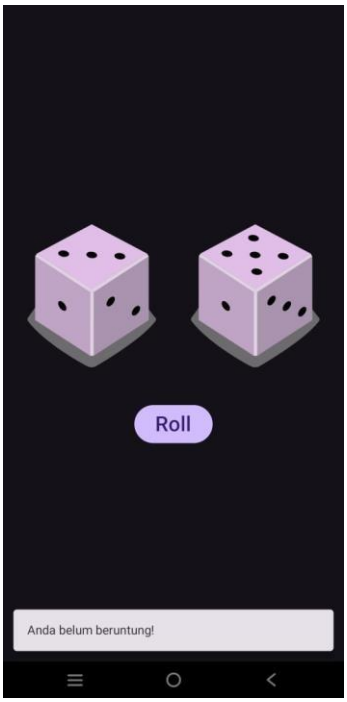
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol "Roll". Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



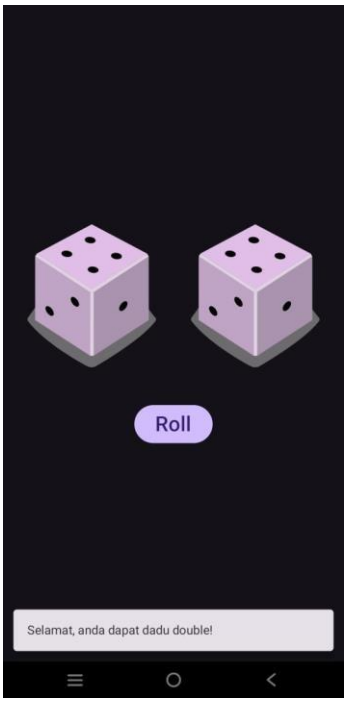
Gambar 1.Tampilan Awal Aplikasi

2. Setelah user menekan tombol "Roll" maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan "Anda belum beruntung!" seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

Tabel 1. Source Code MainActivity.Kt XML

```
package com.example.diceroller_xml

import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.LinearLayout
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.snackbar.Snackbar

class MainActivity : AppCompatActivity() {

    lateinit var diceImage: ImageView
    lateinit var diceImage2: ImageView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val rollButton: Button =
findViewById(R.id.roll_button)
        rollButton.setOnClickListener {
            rollDice()
        }
        diceImage = findViewById(R.id.dice_image)
        diceImage2 = findViewById(R.id.dice_image2)
    }

    private fun rollDice() {
        val result1 = (1..6).random()
        val imageResource1 = when (result1) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }

        diceImage.setImageResource(imageResource1)
        diceImage.contentDescription = result1.toString()

        val result2 = (1..6).random()
        val imageResource2 = when (result2) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
```

```
            else -> R.drawable.dice_6
        }

        diceImage2.setImageResource(imageResource2)
        diceImage2.contentDescription = result2.toString()

        val message = if (result1 == result2) getString(R.string.the_double)
        else getString(R.string.unlucky)

        val layout: LinearLayout = findViewById(R.id.layout)
        val snackbar = Snackbar
            .make(layout, message, Snackbar.LENGTH_LONG)
        snackbar.show()
    }
}
```

Tabel 2. Source Code activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="center_vertical"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:orientation="horizontal"

        <ImageView
            android:id="@+id/dice_image"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
```

```
23
    android:contentDescription="@string/default_content_descr
    iption"
24              app:srcCompat="@drawable/dice_1"
25              tools:srcCompat="@drawable/dice_1" />
26
27          <ImageView
28              android:id="@+id/dice_image2"
29              android:layout_width="wrap_content"
30              android:layout_height="wrap_content"
31
    android:contentDescription="@string/default_content_descr
    iption"
32              app:srcCompat="@drawable/dice_1"
33              tools:srcCompat="@drawable/dice_1" />
34
35      </LinearLayout>
36
37      <Space
38          android:layout_width="0dp"
39          android:layout_height="16dp"/>
40
41      <Button
42          android:id="@+id/roll_button"
43          android:layout_width="wrap_content"
44          android:layout_height="wrap_content"
45          android:layout_gravity="center_horizontal"
46          android:text="@string/roll" />
47  </LinearLayout>
```

Tabel 3. Source Code MainActivity.Kt Compose

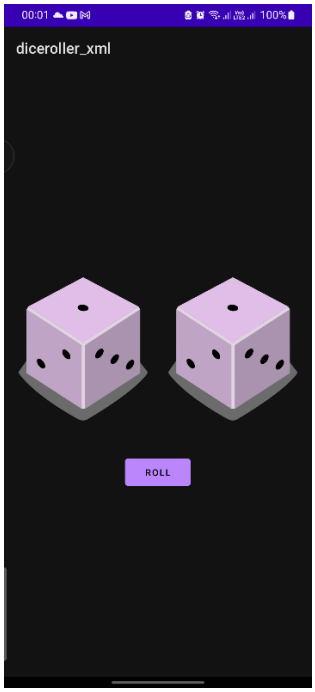
```
1   package com.example.diceroller
2
3   import android.os.Bundle
4   import androidx.activity.ComponentActivity
5   import androidx.activity.compose.setContent
6   import androidx.activity.enableEdgeToEdge
7   import androidx.compose.foundation.Image
8   import androidx.compose.foundation.layout.Column
9   import androidx.compose.foundation.layout.Row
10  import androidx.compose.foundation.layout.Spacer
11  import androidx.compose.foundation.layout.fillMaxSize
12  import androidx.compose.foundation.layout.height
13  import androidx.compose.foundation.layout.padding
14  import androidx.compose.foundation.layout.wrapContentSize
```

```
15 import androidx.compose.material3.Button
16 import androidx.compose.material3.Scaffold
17 import androidx.compose.material3.SnackbarHost
18 import androidx.compose.material3.SnackbarHostState
19 import androidx.compose.material3.Text
20 import androidx.compose.runtime.Composable
21 import androidx.compose.runtime.remember
22 import androidx.compose.runtime.getValue
23 import androidx.compose.runtime.mutableStateOf
24 import androidx.compose.runtime.rememberCoroutineScope
25 import androidx.compose.runtime.setValue
26 import androidx.compose.ui.Alignment
27 import androidx.compose.ui.Modifier
28 import androidx.compose.ui.res.painterResource
29 import androidx.compose.ui.res.stringResource
30 import androidx.compose.ui.tooling.preview.Preview
31 import androidx.compose.ui.unit.dp
32 import com.example.diceroller.ui.theme.DiceRollerTheme
33 import kotlinx.coroutines.launch
34
35 class MainActivity : ComponentActivity() {
36     override fun onCreate(savedInstanceState: Bundle?) {
37         super.onCreate(savedInstanceState)
38         enableEdgeToEdge()
39         setContent {
40             DiceRollerTheme {
41                 DiceRollerApp()
42             }
43         }
44     }
45 }
46
47 @Preview
48 @Composable
49 fun DiceRollerApp() {
50     DiceWithButtonAndImage(modifier = Modifier
51         .fillMaxSize()
52         .wrapContentSize(Alignment.Center)
53     )
54 }
55
56 @Composable
57 fun DiceWithButtonAndImage(modifier: Modifier = Modifier)
   {
58     val scope = rememberCoroutineScope()
```

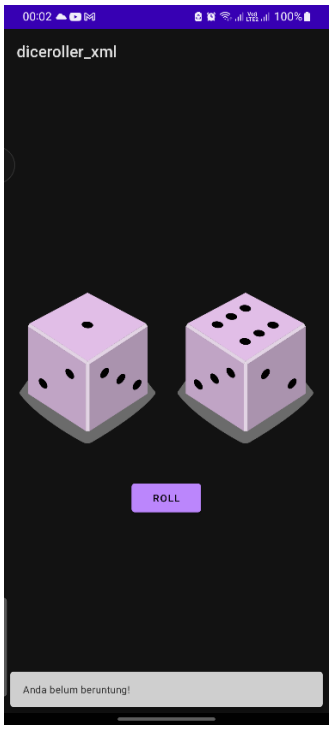
```
    val snackbarHostState = remember { SnackbarHostState() }

    val doubleMessage = stringResource(R.string.the_double)
    val unluckyMessage = stringResource(R.string.unlucky)

    var result1 by remember { mutableStateOf(1) }
    val imageResource1 = when (result1) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    var result2 by remember { mutableStateOf(1) }
    val imageResource2 = when (result2) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Scaffold(
        snackbarHost = {
            SnackbarHost(snackbarHostState)
        }
    ) { paddingValues ->
        Column(
            modifier = modifier.padding(paddingValues),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Row {
                Image(
                    painter = painterResource(imageResource1),
                    contentDescription = result1.toString()
                )
                Image(
```

```
                painter = painterResource(imageResource2),
                contentDescription = result2.toString()
            )
        }

        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = {
            result1 = (1..6).random()
            result2 = (1..6).random()
            if (result1 == result2)
                scope.launch {snackbarHostState.showSnackbar(doubleMessage)}
            else
                scope.launch {snackbarHostState.showSnackbar(unluckyMessage)}
        }) {
            Text(stringResource(R.string.roll))
        }
    }
}
```

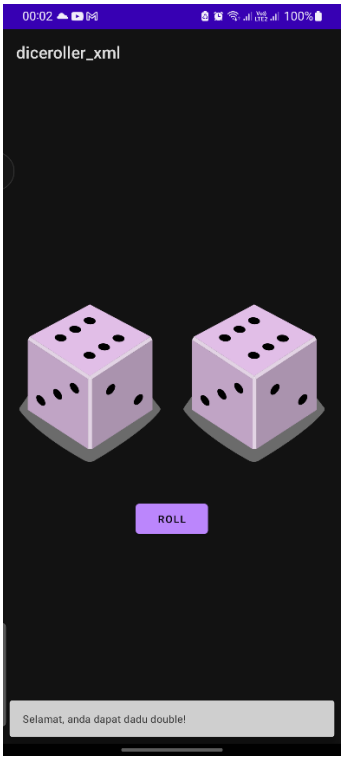
B. Output Program



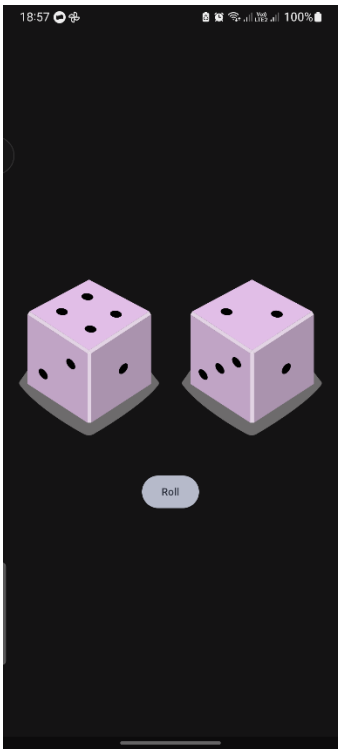
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML



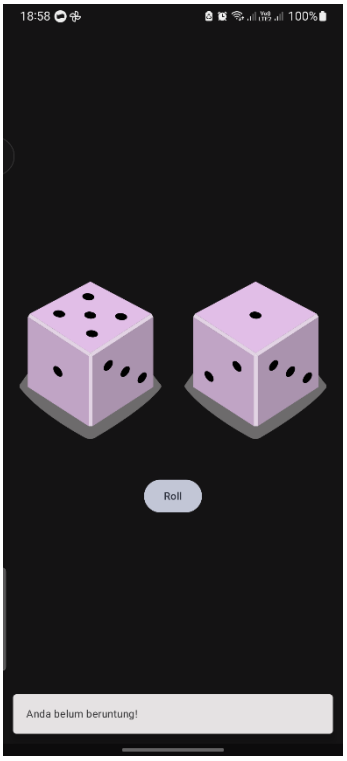
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



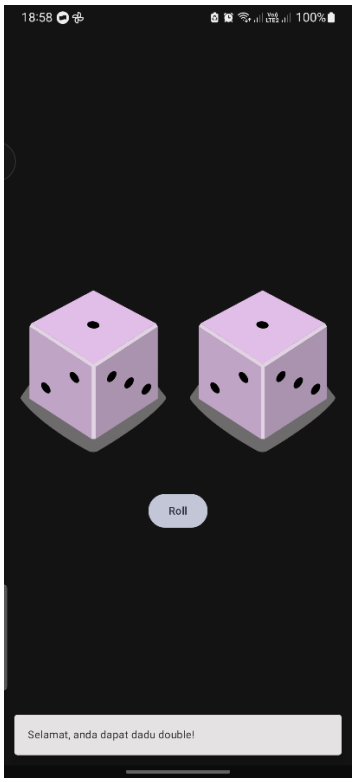
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML



Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.Kt XML

Pada file MainActivity.Kt di project file XML, terdapat class MainActivity yang memuat satu tampilan layar UI dari aplikasi Android. Class ini mewarisi AppCompatActivity() yang berfungsi sebagai backwards compatibility untuk aplikasi Android serta tools untuk manipulasi UI. Kemudian, terdapat deklarasi variabel mutable yang akan diinisialisasi sesudah layout telah diinisialisasi dengan nama diceImage yang digunakan untuk memuat gambar dadu.

Kemudian, terdapat method onCreate() yang merupakan method override dari parent class FragmentActivity.java yang berfungsi untuk melakukan inisialisasi tampilan layar UI. Method ini memanggil parent method onCreate() yang berfungsi untuk menyimpan data kondisi tampilan layar UI. Method setContentView() berfungsi untuk mengatur tampilan UI dari file XML activity_main.xml.

Inisialiasi tombol sebagai variabel immutable dengan nama rollButton dilakukan dengan mengambil id roll_button pada file XML. Kemudian, terdapat method setOnClickListener yang menjalankan kode yang terdapat di dalam method tersebut berupa method rollDice yang berfungsi untuk logika roll serta menampilkan snackbar. Pada method ini, variabel diceImage diinisialisasi dengan mengambil id dice_image pada file XML.

Method rollDice() dengan accessor private dibuat untuk menentukan nilai dari dadu dan mengubah gambar sesuai dengan nilai tersebut dengan mengambil nilai acak dari 1 hingga 6 kemudian mengganti gambar tersebut dengan gambar dadu dengan sisi satu (dice_1) hingga sisi enam (dice_6) dan memuat deskripsi gambar dari hasil acak nilai yang dikonversi ke string.

Logika snackbar dibuat dengan menentukan pesan yang akan ditampilkan yang dimuat pada variabel immutable message dengan melakukan perbandingan antara nilai dadu pertama dan kedua. Jika nilai tersebut sama, maka ambil pesan yang terdapat pada strings.xml dengan nama the_double dan masukkan ke dalam variabel message. Jika nilai berbeda, ambil pesan dengan id unlucky dan masukkan ke dalam variabel message. Snackbar dibuat dengan view yang diambil dari layout yang dibuat pada file XML, teks dari variabel message, dan durasi panjang. Setelah inisialisasi, snackbar ditampilkan dengan method show().

activity_main.xml

Pada file activity_main.xml di project file XML, terdapat tag <LinearLayout> yang berfungsi untuk menyusun view dalam satu kolom atau satu baris. LinearLayout memiliki beberapa attribut seperti id berupa layout, layout_width (lebar layout) yang menyesuaikan parent view (layar keseluruhan tampilan Android), layout_height (tinggi layout) yang menyesuaikan view itu sendiri, layout_gravity yang mengatur posisi child view di posisi tengah secara vertikal, orientation (orientasi layout) vertikal, dan file Kotlin yang berasosiasi dengan layout ini.

Kemudian, terdapat child view <LinearLayout> dengan lebar layout menyesuaikan view itu sendiri, tinggi layout menyesuaikan parent view, orientasi view horizontal, dan posisi layout child view (layout_gravity) tengah horizontal. Child view dari LinearLayout ini adalah dua <ImageView> yang berfungsi untuk memuat gambar. View ini memiliki id dice_image dan dice_image2, panjang dan tinggi layout gambar menyesuaikan gambar itu sendiri, deskripsi gambar yang diambil dari strings.xml dengan nama default_content_description, dan sumber gambar yang diambil dari drawable.xml dengan id dice_1.

View <Space> berfungsi untuk memberikan jarak dengan jarak horizontal 0dp dan jarak vertikal 16dp. View <Button> berfungsi untuk membuat tombol dengan id roll_button, panjang dan tinggi layout tombol menyesuaikan tombol itu sendiri, posisi child view berupa teks berada di tengah horizontal, dan teks yang diambil dari strings.xml dengan nama roll.

MainActivity.Kt Compose

Pada file MainActivity.Kt di project file Compose, terdapat class MainActivity yang memuat satu tampilan layar UI dari aplikasi Android. Class ini mewarisi ComponentActivity() yang berfungsi sebagai komponen utama Jetpack Compose dan library modern yang digunakan Jetpack Compose.

Kemudian, terdapat method onCreate() yang merupakan method override dari parent class ComponentActivity.java yang berfungsi untuk melakukan inisialisasi tampilan layar UI. Method ini memanggil parent method onCreate() yang berfungsi untuk menyimpan data kondisi tampilan layar UI. Method enableEdgeToEdge() berfungsi agar aplikasi Android dapat di-render dan ditampilkan dibelakang navigation bar dan status bar. Method setContent berfungsi untuk mengatur tampilan UI yang diambil dari method composable DiceRollerApp() dan tema aplikasi diambil dari Theme.kt

DiceRollerApp() merupakan method composable yang dapat dipreview di Android Studio yang mengambil view dari method composable DiceWithButtonAndImage() dengan modifier pada method tersebut digunakan untuk mengatur layout UI agar UI menggunakan ukuran semaksimal mungkin dan ukuran child view menyesuaikan ukurannya sendiri dengan orientasi view berada di tengah.

Method composable DiceWithButtonAndImage() berfungsi untuk membuat UI tombol dan gambar. Komponen UI terdiri dari Scaffold yang memuat layout keseluruhan aplikasi, view Column yang memuat child view secara vertikal dengan modifier yang menyesuaikan view Scaffold dan orientasi horizontal view berada di tengah secara horizontal.

Komponen view Row memuat child view secara horizontal dengan child view berupa Image dengan gambar yang diambil dari nilai variabel imageResource yang ditentukan dari nilai acak dari 1 hingga 6. Deskripsi gambar diambil dari nilai hasil acak yang dikonversi ke string. Komponen Space berfungsi untuk membuat jarak antara Row dan Button dengan jarak 16dp

Komponen view Button berfungsi untuk membuat tombol dengan method onClick yang menjalankan kode yang terdapat di dalam method tersebut. Jika ditekan, tombol akan mengambil dua nilai acak dari 1 hingga 6, kemudian melakukan perbandingan antara kedua nilai tersebut. Jika kedua nilai sama, tampilkan snackbar dengan pesan yang diambil dari strings.xml dengan nama the_double. Sebaliknya, tampilkan snackbar dengan pesan dari strings.xml dengan nama unlucky. Komponen text yang merupakan child view Button berfungsi untuk menampilkan teks yang diambil dari strings.xml dengan nama roll.

D. Essay

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pengembangan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.

Jawab:

Pemrogaman imperatif adalah paradigma pemrograman dimana suatu kondisi dapat dimodifikasi melalui pernyataan secara langsung dan berfokus pada bagaimana suatu masalah harus diselesaikan. Pada pemrograman imperatif di UI, *programmer* menentukan tahapan-tahapan yang dicapai untuk membuat suatu komponen UI [1].

Sedangkan, pemrograman deklaratif adalah paradigma pemrograman yang berfokus pada apa yang harus dilakukan dan apa hasil yang diinginkan, kemudian sistem akan bekerja untuk mencapai hasil tersebut. Pada pemrograman deklaratif, *programmer* menentukan bagaimana hasil akhir UI yang diinginkan [1].

Pada konteks pemrograman di Android, pengembangan UI dapat dilakukan menggunakan bahasa Java atau Kotlin dengan pendekatan paradigma imperatif diterapkan menggunakan *Extensible Markup Language* (XML), sedangkan pendekatan paradigma deklaratif diterapkan dengan bahasa Kotlin menggunakan Jetpack Compose [1].

Jetpack Compose adalah tools modern untuk membuat UI Aplikasi Android yang diimplementasikan dengan pendekatan deklaratif yang memudahkan *developer* untuk menentukan komponen UI dan fungsinya tanpa perlu menentukan detail implementasi teknis. Compose memanfaatkan banyak fitur Kotlin melalui fungsi-fungsi *composable* ditandai dengan anotasi `@Composable` yang memberitahu compiler bahwa fungsi tersebut merupakan bagian dari UI dan dikelola oleh *framework* Compose [1].

Aplikasi yang dibangun dengan XML memiliki prinsip navigasi yang mirip dengan Jetpack Compose, tetapi implementasinya berbeda. Layout navigasi pada aplikasi berbasis XML didefinisikan di dalam file XML (contoh: activity_main.xml), sedangkan logika navigasi aplikasi tersebut didefinisikan di dalam file Class Kotlin terpisah (contoh: MainActivity.Kt). Di dalam Class Kotlin tersebut, pernyataan imperatif didefinisikan untuk menampilkan UI yang sesuai [1].

Pada konteks pemrograman di iOS, pengembangan UI melalui pendekatan pemrograman imperatif dilakukan melalui UIKit yang telah digunakan pada iOS selama lebih dari satu dekade. Sementara, pengembangan UI melalui pendekatan pemrograman deklaratif dilakukan melalui SwiftUI yang diperkenalkan pada tahun 2019 yang menandai perubahan mendasar menuju pengembangan UI secara deklaratif [2].

UIKit menerapkan pendekatan imperatif dengan arsitektur *event-driven* dimana *developer* bertanggung jawab mengelola siklus hidup UI (*UI lifecycle events*), perubahan kondisi (*state propagation*), dan pembaruan tampilan (*view update*). Unit arsitektur inti pada UIKit adalah `UIView Controller` yang berperan sebagai perantara antara hierarki *view* dan logika aplikasi [2].

SwiftUI menerapkan pendekatan deklaratif dengan arsitektur *event-driven* yang mengubah secara dasar bagaimana UI dikembangkan dan diperbarui. Pada SwiftUI, suatu *view* diekspresikan sebagai tipe nilai yang *lightweight* yang mendeskripsikan bagaimana suatu UI terlihat pada kondisi tertentu dibandingkan bagaimana UI tersebut seharusnya diperbarui [2].

Berdasarkan riset yang telah saya lakukan, saya lebih prefer paradigma pemrograman deklaratif dikarenakan implementasinya yang lebih sederhana serta lebih *straightforward* dibandingkan paradigma imperatif dimana implementasinya lebih kompleks dan *low-level*.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java?

Jawab:

Dari segi OOP, terdapat perbedaan antara OOP pada Kotlin dan OOP pada Java. Pada deklarasi Class dan enkapsulasi, Java memerlukan d setter dan getter untuk satu atribut. Perlunya setter dan getter pada Java membawa banyak boilerplate pada kode. Sedangkan pada Kotlin deklarasi setter dan getter tidak diperlukan, sehingga nilai dari atribut dapat diakses dan dimodifikasi secara langsung. Tetapi, logika getter pada nilai immutable dan setter pada nilai mutable pada Kotlin tetap terimplementasikan [3].

Implementasi Interface pada Kotlin sama dengan Java, perbedaannya adalah bagaimana method override dilakukan pada kedua bahasa tersebut. Pada Java, method override ditandai dengan annotasi `@Override` pada method yang di-override, sedangkan pada Kotlin langsung menggunakan sintaks `override` pada method yang di-override. Selain itu, implementasi pada interface hanya bisa dilakukan pada Kotlin, dimana implementasi tersebut dapat dipanggil pada class yang menerapkan interface tersebut. Sedangkan, pada Java, implementasi hanya bisa pada method yang di-override. Selain itu, implementasi Abstract Class pada Kotlin juga sama dengan Java [3].

Pada Kotlin, secara default, subclass dari suatu base class tidak bisa dibuat dan method pada base class tidak bisa di-override dikarenakan seluruh class dan method bersifat final, kecuali pada Abstract Class dan Abstract Method. Berbeda dengan Java, dimana subclass dapat dibuat dari base class dan dapat override method apa pun. Inheritance pada Kotlin dapat dilakukan dengan modifier `open` dimana class dengan modifier ini dapat diwarisi oleh subclass. Selain itu, method yang di-override secara default bersifat open dan bisa di-override, dan modifier `final` bisa digunakan untuk melarang method overriding tersebut. [3].

Access modifier atau visibility modifier pada Kotlin sama dengan Java, tetapi pada Kotlin method atau class tanpa visibility modifier secara default bersifat public. Sedangkan, pada Java, method atau class tanpa visibility modifier secara default bersifat package-private, dimana class atau method tersebut hanya bisa diakses dalam package yang sama. Visibility modifier package-private pada Java tidak ada pada Kotlin, tetapi Kotlin memiliki konsep visibility modifier internal dimana method atau class hanya bisa diakses pada satu module, dimana module didefinisikan sebagai kumpulan file Kotlin yang di-compile secara bersama, folder source Gradle, Maven Project, dan modul/folder IntelliJ IDEA [3].

3. Sebutkan, dan jelaskan mengenai SOLID Principles

Jawab:

SOLID principles adalah suatu prinsip atau cara pengelompokan suatu fungsi atau data menjadi suatu class dimana class tersebut memiliki cara tertentu agar bisa saling berhubungan. Tujuan dari SOLID principles adalah untuk mengembangkan struktur sistem pada software yang dapat menerima perubahan dengan mudah, mudah dibaca dan dipahami, serta menjadi komponen dasar yang dapat digunakan pada sistem-sistem lainnya pada suatu sistem software. SOLID principle merupakan singkatan dari lima prinsip-prinsip software design, yaitu Single Responsibility Principle, Open Closed Principle, Liskov Subtitution Principle, Interface Segregation Principle, dan Dependency Inversion Principle [4].

Single Responsibility Principle (SRP) didefinisikan dengan "*A module should have one, and only one, reason to change.*" yang artinya suatu modul harus memiliki satu, dan hanya satu alasan untuk diubah, dimana kata '*module*' yang dimaksud adalah source file atau kumpulan fungsi dan struktur data yang saling berkaitan. Contoh umum pelanggaran SRP adalah class Employee yang memiliki tiga method, yaitu calculatePay(), reportHours(), dan save() (pada kasus ini, terdapat 3 alasan untuk diubah). Permasalahan ini dapat menyebabkan duplikasi kode secara tidak sengaja. Penangan permasalah yang terdapat pada kasus tersebut adalah pemisahan fungsi dan data dimana masing-masing method dipisah menjadi class yang memiliki akses ke data [4].

Open Closed Principle (OCP) didefinisikan dengan "*A software artifact should be open for extension but closed for modification.*" yang dimana maksudnya adalah suatu fungsi (behaviour) pada komponen software dapat diektensi tanpa melakukan perubahan pada komponen software tersebut. Perubahan yang dimaksud adalah bagaimana kode yang telah dibuat akan berubah atau tidak. Arsitektur software yang ideal akan berusaha untuk mengurangi perubahan pada kode seminimal mungkin dengan memisahkan komponen yang dapat diubah dengan beberapa alasan tertentu kemudian mengorganisir dependensi komponen-komponen tersebut dengan sesuai [4].

Liskov Subtitution Principle didefinisikan oleh Barbara Liskov dengan pernyataan "*What is wanted here is something like the following substitution property: If for each object o_1 of type S there is an object o_2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o_1 is substituted for o_2 then S is a subtype of T*". Maksud dari pernyataan tersebut adalah suatu parent class harus memiliki method yang dapat disubtitusikan kepada child class tanpa menimbulkan permasalahan. Contoh permasalahan LSP yang terkenal adalah masalah rectangle/square dimana jika segi empat merupakan subclass dari persegi panjang, maka terdapat permasalahan pada program dikarenakan program mengira bahwa hasil luas yang diambil adalah luas persegi panjang [4].

Interface Segregation Principle (ISP) didefinisikan dengan penggunaan method yang tidak diperlukan tetapi diharuskan pada suatu class yang menerapkan interface tertentu. Sebagai contoh, jika terdapat interface OPS dengan method op1(), op2(), dan op3() yang diterapkan oleh class User1, User2, dan User3, maka masing-masing class User harus menerapkan ketiga method tersebut, dimana seharusnya masing-masing class hanya perlu menerapkan satu method saja. Untuk mengatasi permasalahan tersebut, interface dipecah menjadi 3 bagian sesuai dengan keperluan class, yaitu U1Ops dengan method op1() untuk User1, U2Ops dengan method op2() untuk User2, dan U3Ops dengan method op3() untuk User3 [4].

Dependency Inversion Principle (DIP) didefinisikan dengan suatu kode atau class yang harusnya memiliki dependensi terhadap abstraksi, bukan implementasi konkrit, dikarenakan suatu abstraksi seperti interface atau abstract class jarang mengalami perubahan dibandingkan dengan implementasi konkrit. Penerapan DIP dapat dilakukan dengan mengimplementasikan interface, tidak melakukan inheritance dari class konkrit, tidak melakukan overriding pada method konkrit, dan tidak meng-import class atau method yang bersifat konkrit [4].

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

Jawab:

Di Kotlin, data class secara utama digunakan untuk menyimpan data dimana compiler secara otomatis mengenerasi member functions yang dapat digunakan untuk mencetak data ke format yang bisa dibaca dengan toString(), membandingkan objek atau instance dengan equals() atau hashCode(), serta membuat salinan dari class tersebut dengan copy() [5]. Sedangkan class berfungsi sebagai blueprint atau cetak biru untuk objek yang dibuat jika terdapat logika atau fungsi kompleks. Class tidak memiliki member functions seperti pada data class [6].

E. Daftar Pustaka

[1] N. Nurhadi dan A. Muslim, "Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method," *Journal Research of Social Science, Economics, and Management,* vol. V, pp. 5234-5248, 2025.

[2] M. L. Gondi, "A Comparative Study on the Adoption of Swiftui Over Uikit in Modern IOS Application Development," *International Journal of Research and Scientific Innovation,* vol. XIII, no. 1, pp. 2321-2705, 2026.

[3] S. Aigner, R. Elizarov, S. Isakova dan D. Jemerov, Kotlin in Action, Second Edition, New York: Manning Publications, 2024.

[4] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Boston: Prentice Hall, 2018.

[5] JetBrains, "Data Classes," [Online]. Available: https://kotlinlang.org/docs/data-classes.html. [Diakses 4 April 2026].

[6] JetBrains, "Classes," [Online]. Available: https://kotlinlang.org/docs/classes.html. [Diakses 4 April 2026].

Tautan GIT

Berikut adalah tautan GIT untuk Praktikum Pemrograman Mobile

https://github.com/Arz-zrs/Pemrograman-Mobile